Experimental Characterization of LoRa Propagation and Machine Learning Approaches for Path Loss Modeling Across Diverse Hill Terrains in Nepal

Peter Karki
ST7072CEM Machine Learning
Softwarica College of IT and E-Commerce
Coventry ID: 17624929
260054@softwarica.edu.np

***Abstract*—LoRa could support sensing in Nepal's hills, but most path loss models come from flat or urban areas. We built a two-node 866 MHz instrument that, at each stop, sweeps six spreading factors (SF7–SF12) and three transmit powers (2, 8 and 14 dBm) with five packets each, and collected 61 complete sweeps (5,490 packet slots) in three hill-terrain case studies near Kathmandu. Models were validated grouped by stop and by leaving out a whole case. A depth-3 decision tree using only spreading factor and power predicted delivery with a balanced accuracy of 0.68, and 0.67 on an unseen case. Path loss was not predictable from distance or endpoint terrain features: no model beat the mean-only baseline (RMSE 11.1 dB), and a log-distance fit held only in the short-range case ($n = 2.10$). The cheapest setting delivering four of five packets needed a median 97% less transmit energy than SF12 at 14 dBm. An interactive dashboard maps predicted dead zones on example terrain. In hill terrain, what lies between the antennas matters more than the distance between them.**

***Index Terms*—LoRa, path loss, machine learning, hill terrain, grouped cross-validation, Nepal**

I. Introduction

LoRa is a long-range, low-power radio widely used for Internet of Things (IoT) sensing. In Nepal's hills it could support agriculture and hazard monitoring where cellular coverage is patchy. Planning such networks needs an estimate of path loss, usually from the log-distance model, in which loss grows with the logarithm of distance [1]. Its parameters, however, come mostly from flat or urban areas. In hills, two points a few hundred metres apart may be separated by a ridge, while points kilometres apart across a valley may see each other clearly. Planners therefore rely on trial and error, and field data from such terrain are scarce for Nepal.

Machine learning (ML) can add terrain information to path loss prediction, but two problems limit how far published results transfer: few studies cover hill terrain in South Asia, and random train–test splits can put measurements from the same place in both sets and overstate accuracy. We therefore built an instrument that tests every combination of six spreading factors and three transmit powers at each stop, which separates the effect of radio settings from that of location, and measured three contrasting case studies (Table I). The research questions are:

TABLE I: The three case studies

Case	Site	Setting	Range	Stops
1	A	Forested slope; Station antenna 2.0 m	22–250 m	17
2	B	Hill slope across a valley; antenna 1.0 m	0.2–2.9 km	36
3	C, D	Town rooftops (≈9 and 6 m)	0.3–3.3 km	8

- **RQ1:** Do distance and simple terrain features improve path loss prediction over log-distance or mean-only baselines?
- **RQ2:** Do models trained in some terrain cases transfer to a different case?
- **RQ3:** What is the cheapest radio setting each location needs?

The contributions are:

1) a reproducible two-node instrument that records delivery, RSSI and SNR for 18 radio configurations at every stop, and a dataset of 61 complete sweeps in three hill-terrain cases;
2) an evaluation of regression, decision-tree and ensemble models against simple baselines, using validation grouped by stop and a leave-one-case-out test;
3) evidence that radio settings transfer between terrains while distance-based path loss does not, an explanation of why, and the cheapest working setting at each stop;
4) an interactive dashboard that presents the results and maps predicted dead zones on example terrain.

Table II lists the abbreviations used.

II. Literature Review

A. Classical path loss models

The log-distance model fits a path loss exponent n and a shadowing term to measurements [1]. The Okumura–Hata model adds corrections for frequency and antenna heights, but assumes base stations above 30 m and distances over 1 km [2], and diffraction models such as ITU-R P.526 estimate the extra loss over hills [3]. At 866 MHz these models differ by more

TABLE II: Abbreviations

Abbreviation	Full form
AP / AUC	Average precision / area under the ROC curve
IoT	Internet of Things
GPS	Global Positioning System
LiDAR	Light detection and ranging
LOCO / LOO	Leave-one-case-out / leave-one-stop-out
OLS	Ordinary least squares
LoRa	Long Range (radio modulation)
LPWAN	Low-power wide-area network
ML	Machine learning
NOACK	No acknowledgement (handshake failed)
PL	Path loss
RF	Random forest
RMSE	Root-mean-square error
ROC	Receiver operating characteristic
RQ	Research question
RSSI / SNR	Received signal strength / signal-to-noise ratio
SF	Spreading factor
SRTM	Shuttle Radar Topography Mission

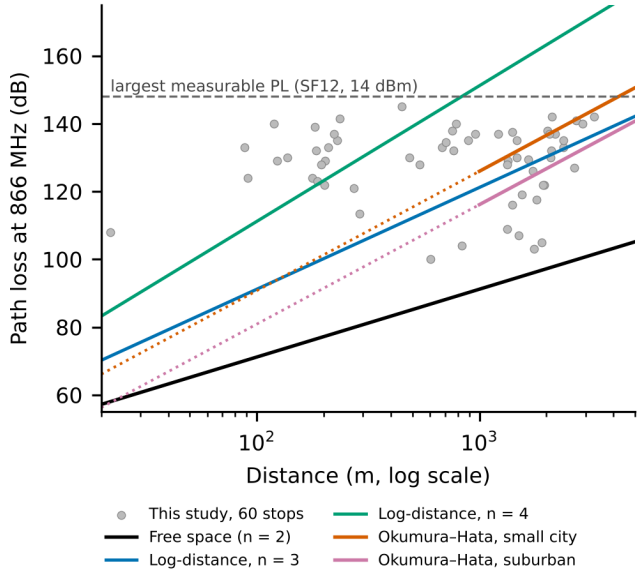


Fig. 1: Classical path loss models at 866 MHz with this study's 60 stops (grey). Drawn by the author from the published equations.

than 40 dB at 1 km, and none follows the spread of our stops (Fig. 1).

B. Empirical LoRa measurements

Petäjäjärvi *et al.* [4] measured LoRa at 868 MHz in Oulu, Finland, and reported ranges over 15 km on flat ground and about 30 km over water at SF12 and 14 dBm (Fig. 2). They also derived a channel attenuation model for similar flat areas.

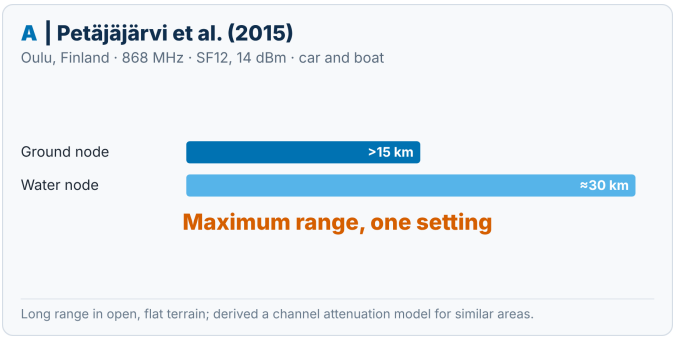


Fig. 2: Conceptual summary of [4]: maximum range with one setting in flat terrain. Drawn by the author.

Rademacher *et al.* [5] collected over 112,000 samples during 230 days in a 200 km^2 urban area and compared eleven models. Log-distance, Okumura and Winner+ gave reasonable estimates, whereas terrain-based models gave no gain (Fig. 3).



Fig. 3: Conceptual summary of [5]: eleven empirical models compared in a city. Drawn by the author.

In an Alpine canyon, Bianco *et al.* [6] measured 868 MHz LoRa with a single setting (SF7, 14 dBm) and reached only about 300 m at 50% delivery, far less than on flat ground (Fig. 4).

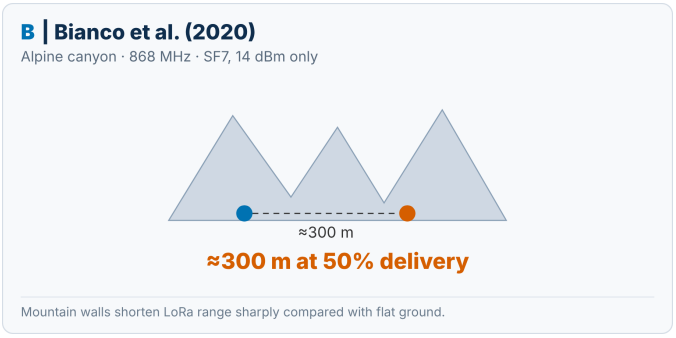


Fig. 4: Conceptual summary of [6]: range at 50% delivery in an Alpine canyon. Drawn by the author.

In Nepal, Pokharel *et al.* [7] recommended settings from one 433 MHz link of about 5 km, without ML (Fig. 5). All four studies used one link or one radio setting at a time,

which motivates multi-configuration measurements at many locations.

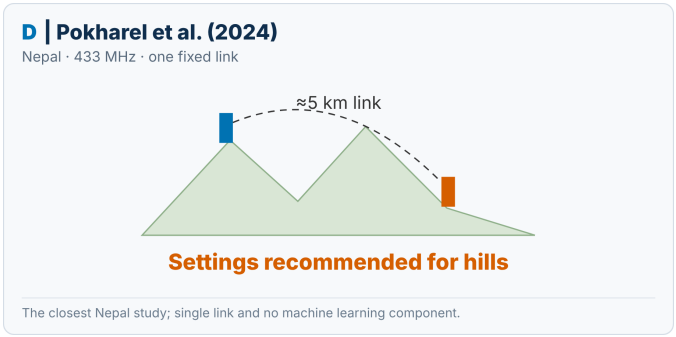


Fig. 5: Conceptual summary of [7]: one long link in Nepal used to recommend settings. Drawn by the author.

C. Machine learning for path loss

Ojo *et al.* [11] fed the sequence of land-cover classes along each link to a bidirectional LSTM and reported a mean error of 4.97 dB from over 35,000 packets of sheep-tracking devices on Alpine pastures, about half the error of established empirical models (Fig. 6).

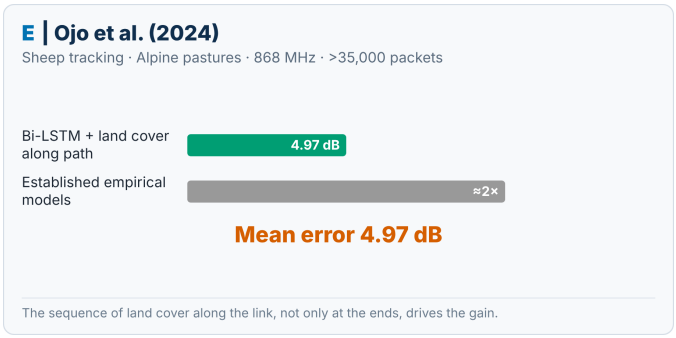


Fig. 6: Conceptual summary of [11]: a sequence model using land cover along the path. Drawn by the author.

Ethier and Châteauvert [15] showed, on 8.2 million measurements, that adding obstacle depth along the path cut the median RMSE by about 5 dB, to 7.5 dB (Fig. 7).

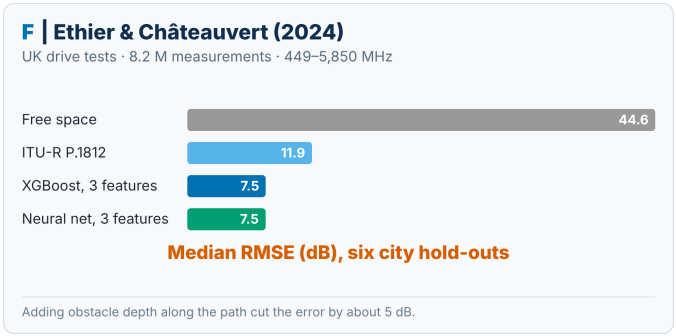


Fig. 7: Conceptual summary of [15]: median RMSE over six held-out cities. Drawn by the author.

Bitterling *et al.* [8] found that random forests [9] with LiDAR terrain features beat baselines under random splits but degraded when whole gateways were held out (Fig. 8). Together these works suggest that terrain *along the path* is informative and that validation design strongly affects reported accuracy. Decision trees [10] remain attractive for small datasets because their rules can be read and checked.

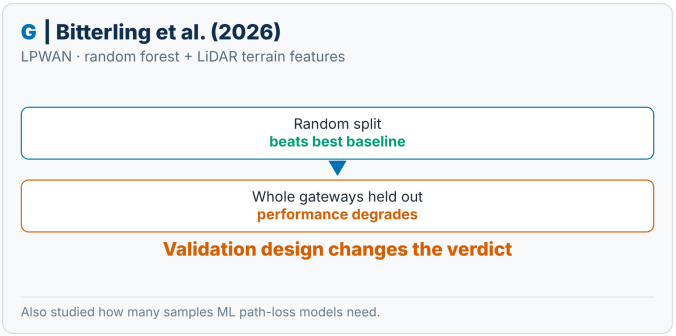


Fig. 8: Conceptual summary of [8]: the verdict changes with the validation design. Drawn by the author.

D. Censoring and validation

Weak links are lost rather than measured, so ignoring them biases the fitted exponent; a Tobit estimator [12] corrects this [13] (Fig. 9). Samples grouped in space also make random cross-validation optimistic [14]; packets from one stop form such a group. LoRa data are strongly affected by both issues because many packets are lost at long range and each location yields many packets.

E. Why the reported metrics are not benchmarks

The studies answer different questions with different data: maximum range, delivery at one setting, mean error of a sequence model, or RMSE over held-out cities. Their frequencies, sample sizes, targets and validation schemes differ, so their numbers describe the field but cannot be ranked against each other or against this study. Table III therefore keeps each metric in its original form.

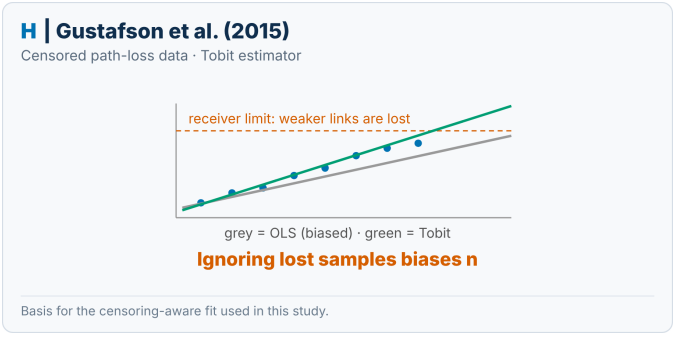


Fig. 9: Conceptual summary of [13]: ignoring lost samples biases the fitted exponent. Drawn by the author.

TABLE III: Comparable studies (metrics in original form; not directly comparable)

Study	Setting	Method	Result	Limitation here
Petäjäjärvi [4]	Flat land and sea	SF12, 14 dBm	>15 km, ≈30 km	Flat; one setting
Rademacher [5]	Urban, 112k samples	11 empirical models	Log-distance adequate	City; no ML
Bianco [6]	Alpine canyon	SF7, 14 dBm	≈300 m at 50%	One setting
Pokharel [7]	Nepal, 433 MHz	One link	Settings advice	Single link
Ojo [11]	Alpine pastures	Bi-LSTM, land cover	4.97 dB mean error	Large dataset
Ethier [15]	UK drive tests	ML, obstacle depth	RMSE ≈7.5 dB	Urban, 8.2M samples
Bitterling [8]	LPWAN gateways	RF + LiDAR	Drops on new gateways	–
This work	Nepal hills, 61 stops	Trees, ridge, Tobit, RF	0.67 on unseen case	–

TABLE IV: Research gaps

Gap	Prior work	This study
Terrain	Flat, urban, Alpine [4]–[6]	Three Nepal hill cases
Settings	One per campaign [6]	18 per stop
Censoring	Rarely handled [13]	Tobit; 30 lost locations
Validation	Random splits [8]	By stop and by case

TABLE V: Sweep configuration

Parameter	Value
Frequency / bandwidth	866.1 MHz / 125 kHz
Spreading factor	SF7–SF12
Transmit power	2, 8, 14 dBm
Packets per setting	5 (90 slots per stop)
Control link	SF12, 14 dBm

F. Research gap

No reviewed study combines multi-setting LoRa measurements in Himalayan foothills, explicit handling of censored and failed links, and ML validation that holds out whole terrain cases (Table IV). This study is small, so it does not aim to outperform the reviewed work; it tests whether their approaches hold in a new setting.

III. Measurement Setup and Data

A. LoRa background

LoRa uses chirp spread spectrum [16], [17]. Each step from SF7 to SF12 doubles airtime and lowers the SNR needed for decoding by about 2.5 dB, down to about -20 dB at SF12. A higher SF therefore trades airtime and energy for range, and links can survive below the thermal noise floor. Each extra decibel of transmit power adds one decibel at the receiver. Testing every SF–power pair at one place shows how much margin a link has.

B. Instrument

Both nodes use an ESP32 and an Ai-Thinker Ra-08H module at 866.1 MHz and 125 kHz, inside the band Nepal assigns to IoT [18] (Figs. 10 and 11). The fixed *Station* transmits; the walking *Roamer* receives and logs every slot with RSSI, SNR and GPS position (5–10 m accuracy). Custom radio firmware was written because the vendor firmware cannot change SF and power directly; it was frozen for all measurements and fingerprinted with SHA-256 so any change would be detectable. The Roamer firmware (version 4.6.0) logs every slot to a CSV file on a microSD card. Antenna gains and cable losses were not measured.

C. Sweep protocol

At each stop the Roamer requests a sweep over a control link (SF12, 14 dBm). The Station then sends five packets for each of 18 configurations, 90 slots in about 67 s, each logged as OK or TIMEOUT (Table V, Fig. 12). If the handshake fails (NOACK), no sweep runs, and the place is recorded as a failed attempt. In the Case 1 logs, sweeps after strong handshakes started 3.5–7 s later and sweeps after weak handshakes 11–46 s later, consistent with link margin rather than a clock fault.

D. Data and cleaning

The raw data are one Roamer log per site. They were combined, and only complete sweeps were kept: 61 stops (17, 36 and 8 per case), 5,490 slots and 2,612 received packets (47.6%). The 56 failed handshakes reduce to 30 unique *lost locations* (3, 9 and 18 per case) after removing retries within 30 m of a completed stop. An earlier Site A run on older firmware was excluded because delivery at the same spots differed markedly. A scripted pipeline rebuilds the stop table (61 rows) and packet table (5,490 rows) from the raw logs and reproduces every reported number.

IV. Methodology

A. Workflow and problem formulation

Fig. 13 summarises the workflow. For stop s and slot k with configuration $c_k = (\mathrm{SF}_k, P_k)$, the targets are

$$y_s = \mathrm{median}\{PL_{s,k} : P_k = 14\,\mathrm{dBm},\ z_{s,k} = 1\}, \quad (1)$$

$$z_{s,k} = 1 \text{ if slot } k \text{ was received, else } 0, \quad (2)$$

$$c_s^* = \arg\min_c \ \mathrm{cost}(c) \ \text{s.t.} \textstyle\sum_{k:c_k=c} z_{s,k} \geq 4, \quad (3)$$

with $PL = P_{tx} - \mathrm{RSSI}$ and $\mathrm{cost}(c) = 10^{P/10} \cdot 2^{\mathrm{SF}-7}$. Path loss y_s is a regression target, delivery $z_{s,k}$ a classification target and c_s^* the cheapest working setting (RQ3). Apparent path loss includes the unmeasured antenna gains. Only 14 dBm packets define y_s because the measured power steps were larger than nominal; 60 stops have a value, as one Case 1 stop received nothing.

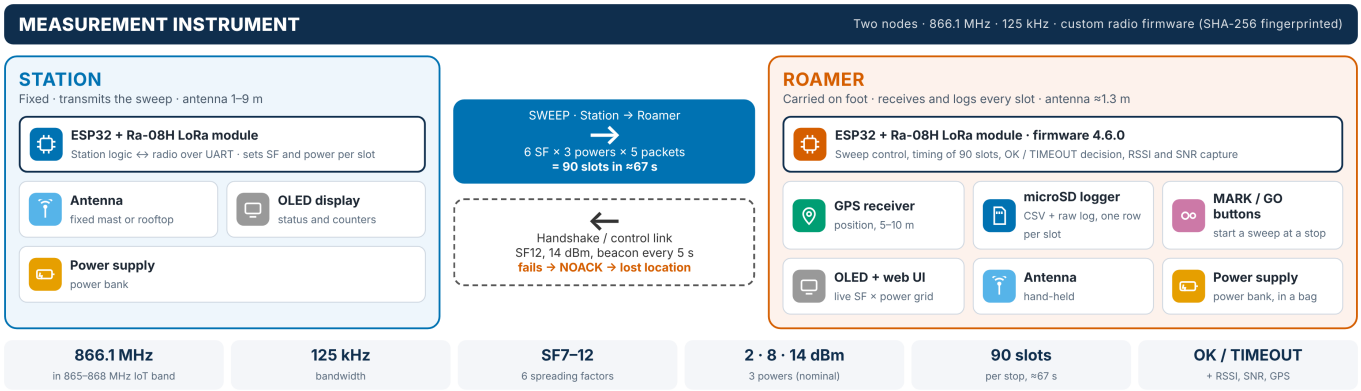
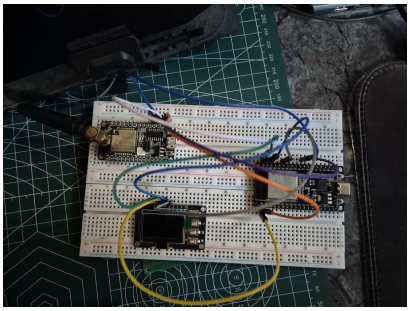
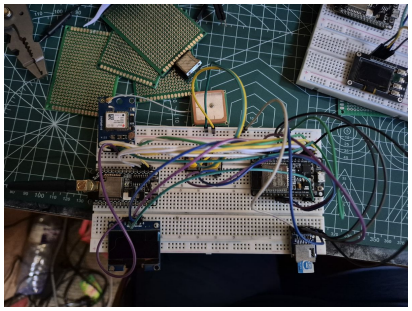


Fig. 10: Hardware of the two-node instrument and its two radio links.



(a) Station



(b) Roamer



(c) Roamer in the field

Fig. 11: The instrument and its use in the field.

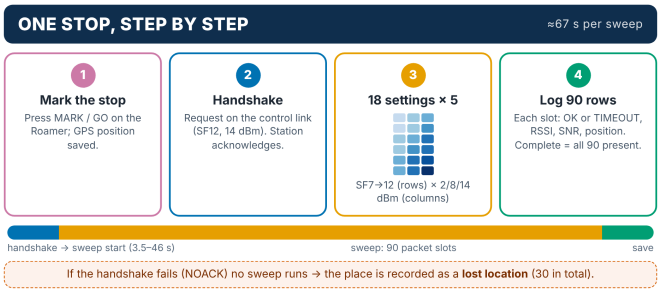


Fig. 12: One stop, step by step.

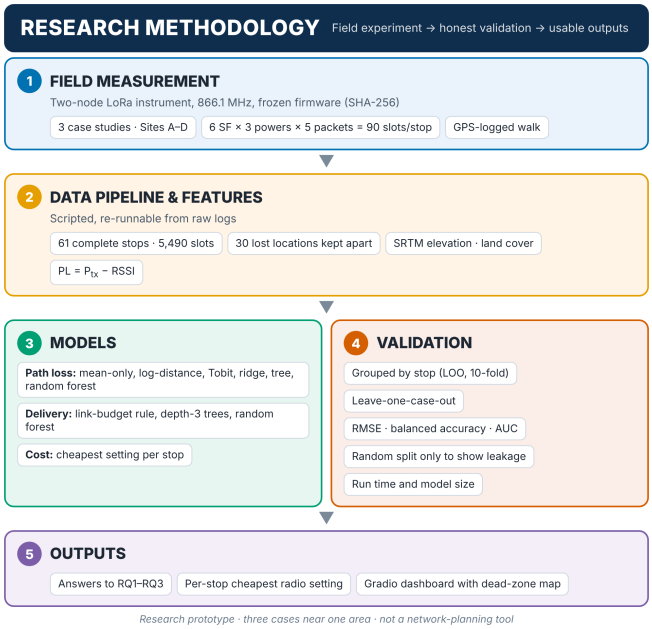


Fig. 13: Research methodology workflow.

B. Features

Distance from Station to stop was computed with the haversine formula. Ground elevations of stops and Stations came from the SRTM 30 m elevation model [19], giving the elevation difference (stop minus Station) and the elevation angle between the antennas. Land cover around each stop was labelled from satellite imagery as forest, built-up, mixed or open (14, 12, 31 and 4 stops) and one-hot encoded with forest as the reference. The models use $\log_{10}$ distance. Station antenna height (one value per site) and a manual line-of-sight label (made with delivery visible) were excluded to avoid leakage.

TABLE VI: Model settings

Model	Setting
Ridge	30 penalties, 10^{-2}–10^{3}
Regression tree	Depth 3, $\geq$5 stops per leaf
Delivery tree	Depth 3, $\geq$90 packets per leaf
Random forest	300 trees
Tobit	L-BFGS-B; 300 bootstrap resamples
Threshold / seed	0.5 / 0

C. Models and why they were chosen

Path loss is continuous, so it is a regression problem: a mean-only baseline; the log-distance model $PL(d) = PL_0 + 10\,n \log_{10} d + X_\sigma$; least squares and ridge regression with terrain features (ridge suits few, correlated features); a depth-3 regression tree; and a 300-tree random forest. A Tobit fit on all packets treats each lost packet as path loss above its measurable limit [12], [13].

Delivery is yes/no, so it is a classification problem. Depth-3 trees [10] with at least 90 packets per leaf were the main classifier: receiver sensitivity behaves like a threshold, the rules are readable, no leaf can memorise one stop, and the model is small enough for a node. Baselines were the majority class and a link-budget rule, which predicts delivery when the transmit power minus the log-distance prediction exceeds the sensitivity of the spreading factor; a random forest [9], logistic regression and gradient boosting tested whether more flexible models help. Its random-split score was also recorded to show the effect of leakage. Neural networks were not used: with 61 independent locations, extra capacity would mainly memorise stops, and the sequence models that work well [11] relied on tens of thousands of links.

D. Validation and metrics

Packets from one stop share location and terrain, so they are not independent [14]. If one stop's packets appeared in both training and test sets, a model could recognise the stop instead of learning a general relationship. Every split therefore keeps a stop together (Fig. 14). Regression used leave-one-stop-out (LOO) and leave-one-case-out (LOCO); classifiers used 10-fold cross-validation grouped by stop and LOCO. Metrics were RMSE, balanced accuracy, precision, recall, F_1, AUC and average precision:

$$\mathrm{RMSE} = \sqrt{\tfrac{1}{N}\textstyle\sum_i (y_i - \hat{y}_i)^2}, \quad F_1 = \tfrac{2PR}{P+R}, \tag{4}$$

$$\mathrm{BalAcc} = \tfrac{1}{2}\left(\tfrac{TP}{TP+FN} + \tfrac{TN}{TN+FP}\right), \tag{5}$$

where $P = TP/(TP + FP)$ and $R = TP/(TP + FN)$. Balanced accuracy is the headline metric because it weighs lost and delivered packets equally (0.5 is chance). With three cases, LOCO results are indicative. Associations used Spearman's ρ and differences between cases the Kruskal–Wallis test. Table VI lists the settings; the analysis used Python and scikit-learn [20].

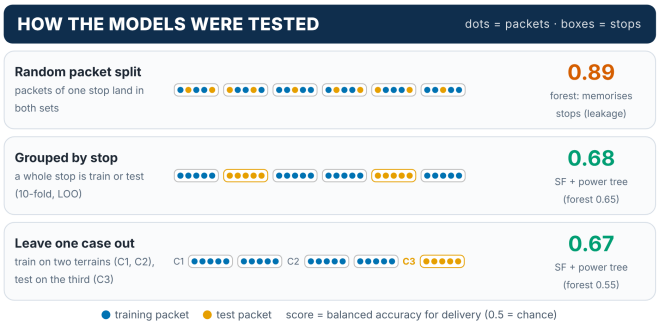


Fig. 14: Validation schemes and resulting balanced accuracy.

E. Computational complexity

For n samples and d features, least squares costs $O(nd^2)$, a tree $O(d\,n \log n)$ to train and $O(\text{depth})$ to predict, a forest of T trees T times more, and Tobit $O(kn)$ for k iterations. Training time, prediction time and model size were measured as medians of five single-thread runs on a 2.1 GHz cloud machine. A depth-3 tree answers with at most three comparisons, which matters on a small node.

F. Dashboard

A Gradio dashboard (Appendix B) shows the results, reruns grouped validation live and maps the tree's predicted dead zones on synthetic hill terrain, beside a geometric line-of-sight layer that the model cannot see. The expected receiver sensitivity used for the instrument check is the thermal noise floor (≈ -117 dBm for 125 kHz and a 6 dB noise figure) plus the LoRa SNR limit of each spreading factor [16].

V. Results

A. Instrument and delivery

The weakest received packets matched the expected sensitivity: -124 dBm at SF7 (expected -124.5) and -134 dBm at SF12 (expected -137) (Fig. 15). The lowest SNRs (-10 to -21 dB) were within about 2.5 dB of the demodulation limits, so the radios behaved as specified. Raising power from 2 to 14 dBm raised the median RSSI at the same stop and SF by 15.3 dB, not the nominal 12 dB (8 to 14 dBm: 8 dB; 2 to 8 dBm: 7 dB), which supports using only 14 dBm packets for path loss. Delivery rose with SF and power in every case, from 16% at SF7/2 dBm to 88% at SF12/14 dBm (Fig. 16). At SF12/14 dBm, delivery was 75%, 92% and 95% in Cases 1–3. Case 1 delivered only 2% at SF7/2 dBm, against 24% in Case 2, consistent with heavy obstruction at short range. Most stops were neither fully connected nor disconnected, the regime in which configuration choices matter most. Differences between cases were not significant (Kruskal–Wallis $p = 0.18$ for delivery, $p = 0.34$ for path loss; Table VII).

B. Path loss, distance and link existence

Pooled over 60 stops, path loss was nearly independent of distance ($n = 0.14$, 95% CI -0.44 to 0.72, $\sigma = 11.1$ dB; Fig. 17, Table VIII). Only Case 1 followed the expected trend

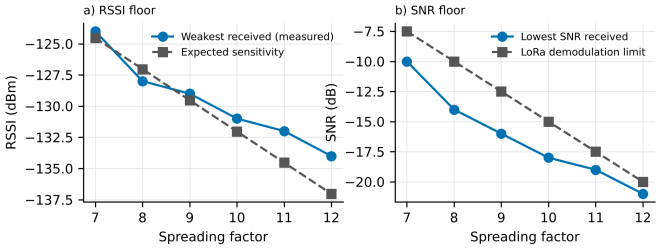


Fig. 15: Weakest received RSSI and lowest SNR per SF against expected limits.

TABLE VII: Case-study profiles

	Case 1	Case 2	Case 3
Stops (sites)	17 (A)	36 (B)	8 (C, D)
Median distance (m)	185	1,374	2,248
Distance range (m)	22–250	209–2,916	272–3,268
Elevation diff. (m)	0 to 36	−34 to 113	−4 to 287
Received / slots	563/1,530	1,737/3,240	312/720
Delivery (%)	36.8	53.6	43.3
Median PL_{14} (dB)	129.5	130.0	134.0
Lost locations	3	9	18
Link success (%)	85	80	31
ρ(delivery, distance)	−0.58*	−0.02	−0.42
ρ(delivery, elev. diff.)	−0.26	0.49*	−0.04

*$p < 0.05$.

($n = 2.10$, $R^2 = 0.41$). Case 2 showed no trend, with path loss at similar distances ranging from about 100 to 145 dB, and the Case 3 fit rests on eight stops with a confidence interval that includes zero. The censoring-aware Tobit fit was also flat ($n = -0.12$, CI -1.12 to 0.64). Delivery rose with the stop's elevation above the Station ($\rho = 0.33$, $p = 0.008$), driven by Case 2 ($\rho = 0.49$), while delivery fell with distance only in Case 1 ($\rho = -0.58$). Counting the 30 lost locations, link success fell from 91% within 300 m to 52% at 2–4 km, and was only 31% in Case 3 (Fig. 18).

C. Path loss regression

No model beat the mean-only baseline (Table IX, Fig. 19). Under LOCO, terrain features raised the error to 15.0–15.8 dB, including for the random forest, against 11.4 dB for the baseline and 13.0 dB for log-distance; the largest errors occurred when Case 3 was held out. Within one case, elevation helped slightly: in Case 1 the LOO RMSE fell from 8.7 to 7.2 dB. Restricting the data to 51 stops with at least ten received packets gave the same picture. The excluded manual line-of-sight label did separate delivery (27%, 53% and 76% for blocked, partly blocked and clear), which shows that path obstruction matters.

D. Delivery classification

A depth-3 tree on SF and power reached a balanced accuracy of 0.68 (AUC 0.71), and 0.67 under LOCO, holding at 0.69, 0.67 and 0.72 for the held-out Cases 1–3 (Table X). Adding terrain features lowered LOCO to 0.50 (Fig. 20). The random forest scored 0.89 under a random packet split but 0.65 grouped and 0.55 on an unseen case: it recognised stops

TABLE VIII: Log-distance fits of median 14 dBm path loss

Fit	PL_0 (dB)	n	95% CI of n	R^2	σ (dB)
All stops (60)	124.5	0.14	−0.44, 0.72	0.00	11.1
Case 1 (16)	84.3	2.10	0.65, 3.56	0.41	6.7
Case 2 (36)	128.8	−0.07	−1.66, 1.53	0.00	12.6
Case 3 (8)	88.0	1.41	−0.15, 2.97	0.45	5.8
Tobit, all packets	140.6	−0.12	−1.12, 0.64	–	17.7
Tobit, Case 1	73.2	3.09	–	–	12.4

TABLE IX: Path loss regression RMSE (dB)

Model	LOO	LOCO	LOCO per case
Mean only	**11.1**	**11.4**	8.4 / 12.9 / 9.1
Log-distance	11.3	13.0	9.8 / 14.8 / 9.5
+ elevation (OLS)	11.4	15.8	11.2 / 16.3 / 20.9
+ land cover (ridge)	11.1	15.7	8.4 / 17.1 / 19.8
Regression tree	11.2	15.3	9.5 / 16.9 / 16.8
Random forest	11.3	15.0	11.8 / 16.5 / 13.0

rather than learning transferable rules. The tree was the most precise model (0.70); the link-budget rule predicted almost every packet as delivered (recall 0.97, precision 0.50). Gradient boosting was marginally best within known cases (AUC 0.75) but fell to 0.58 on an unseen case (Fig. 21). Of 2,878 lost packets, the tree flagged 2,217 correctly, and it found 1,534 of 2,612 delivered ones. A learning curve rose from 0.57 with 9 training stops to 0.62–0.64 with 39–49 stops and then levelled off. The tree fitted to all data splits first on power ($\leq$11 dBm): at low power, delivery is likely only when the stop lies more than about 57 m above the Station, and at 14 dBm SF11–12 delivers most packets (Fig. 22).

E. Cheapest setting and computational cost

At 55 of 61 stops a setting delivered at least four of five packets; 19 needed only SF9 or below at 8 dBm or below, and 7 needed SF11–12 at 14 dBm. The median stop could have used 97% less energy than SF12 at 14 dBm (92% in Case 3). These choices rest on five packets per setting, so they describe what was measured rather than a guaranteed long-term setting. The delivery tree trained in about 1–3 ms and occupied about 2 kB, whereas the forest took 0.86 s and about 24 MB without performing better (Table XI).

VI. Discussion

A. Answers to the research questions

- **RQ1:** Distance and endpoint terrain features did not beat the mean-only baseline; distance mattered only in Case 1 and for whether a link formed at all.
- **RQ2:** Radio-setting models transferred (0.67 on an unseen case); terrain-feature and more flexible models did not.
- **RQ3:** The cheapest setting varied widely between stops, and most stops needed far less energy than the safest setting.

Fig. 23 summarises these findings.

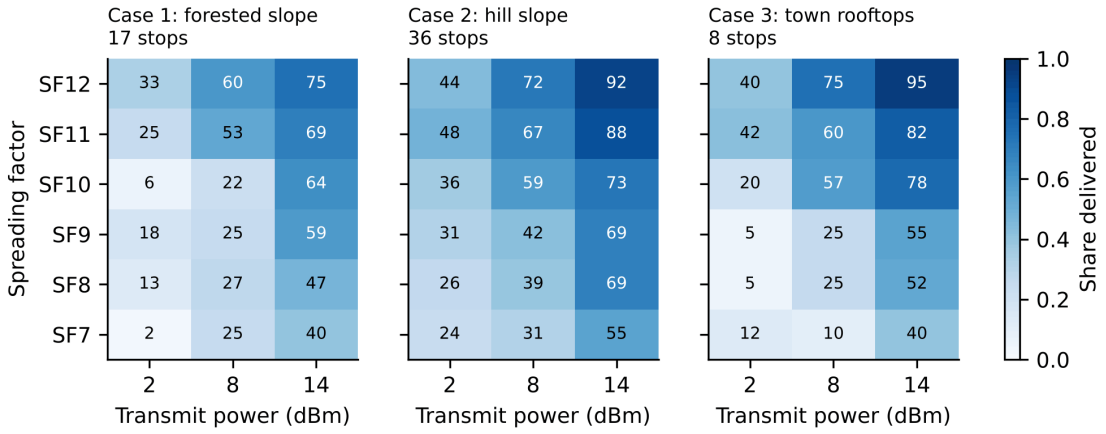


Fig. 16: Share of packets delivered for each SF and transmit power, per case.

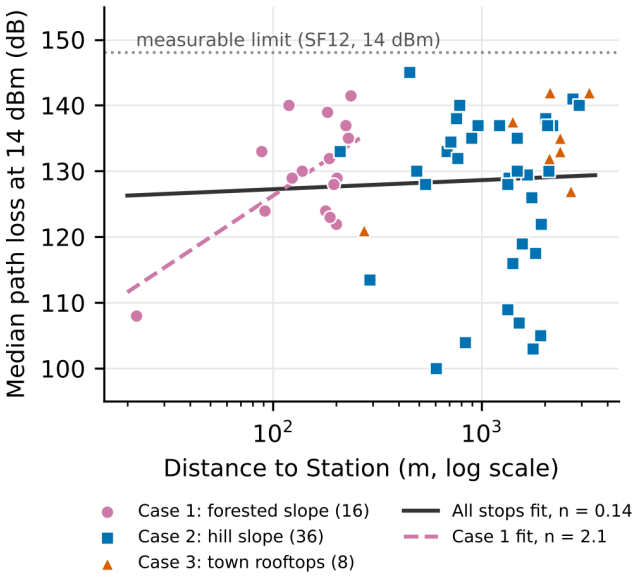


Fig. 17: Median 14 dBm path loss against distance with log-distance fits.

TABLE X: Delivery metrics, 10-fold grouped (threshold 0.5)

Model	BalAcc	Prec.	Rec.	F1	AUC	AP	LOCO
Link-budget rule	0.54	0.50	0.97	0.66	0.73	0.71	–
Tree, SF + power	**0.68**	**0.70**	0.59	0.64	0.71	0.68	**0.67**
Tree, all features	0.62	0.63	0.53	0.57	0.64	0.60	0.50
RF, all features	0.65	0.63	0.62	0.63	0.71	0.66	0.55
Logistic, SF + power	0.68	0.67	0.64	0.65	0.73	0.71	0.65
Logistic, all	0.67	0.66	0.64	0.65	0.73	0.70	0.56
Grad. boosting, all	0.68	0.66	0.67	**0.67**	**0.75**	**0.72**	0.58

B. *Why distance and terrain features failed*

Three effects flatten the pooled distance trend. Censoring removes the weakest packets, although the Tobit fit shows it is not the whole story. Survivorship hides 30 mostly distant locations where no link formed. Including the lost locations,

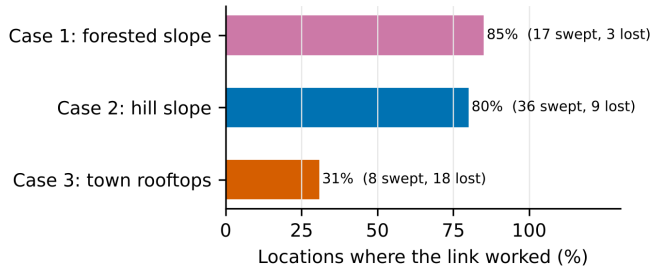


Fig. 18: Share of attempted locations where a sweep was completed.

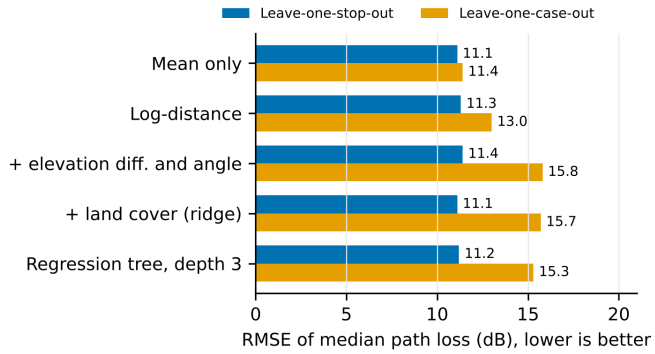


Fig. 19: Path loss RMSE under LOO and LOCO validation.

link success falls from 91% to 52% with distance, a trend the sweep data alone hide. Mixed terrain means distant stops across a valley can see the Station better than nearby stops behind a ridge; the positive link between delivery and stop elevation in Case 2 supports this. Endpoint features describe the two ends of a link but not the terrain between them, which matches the gains reported from path features [11], [15] and the drop on held-out gateways [8].

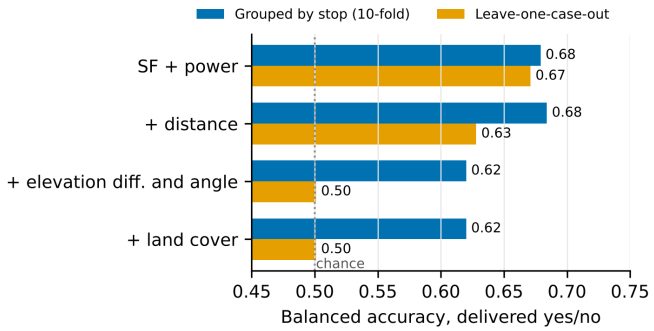


Fig. 20: Balanced accuracy of depth-3 trees for growing feature sets.

TABLE XI: Measured computational cost

Model	Complexity	Fit (ms)	Size (kB)
Log-distance (OLS)	$O(nd^2)$	0.6	0.5
Ridge	$O(a\,nd^2)$	2.2	1.2
Regression tree	$O(d\,n \log n)$	0.5	1.7
RF regressor	$O(T d\,n \log n)$	220	1,581
Tobit	$O(kn)$	28.9	0.2
Tree, SF + power	$O(d\,n \log n)$	1.2	2.3
RF classifier	$O(T d\,n \log n)$	865	24,405

C. Why features did not transfer

Within one case, terrain features helped slightly; across cases they encoded case-specific relationships and raised the error, especially for the small Case 3. A model trained where height above the Station predicted delivery (Case 2) misled it where distance mattered instead (Case 1). This is the same failure Bitterling *et al.* [8] saw when whole gateways were held out, and it is why only small, interpretable models were used.

D. Was the model choice justified?

Yes. Every flexible model lost accuracy on an unseen case, while both SF + power models kept it. The shallow tree matched the best transferable model, was the most precise, used three readable rules and was about 10,000 times smaller than the forest. Regression was the right framing for path loss, but no regression model, simple or flexible, beat the mean: the limit was the information in the features, not the model. With 61 stops, simplicity was a strength rather than a compromise.

E. Comparison and practical use

The Case 1 exponent (2.10) is close to free space, and the shadowing values (5.8–12.6 dB) are typical for outdoor LoRa. Unlike flat-terrain ranges of tens of kilometres [4], 48% of hill locations beyond 2 km formed no link, so a setting recommended from one link [7] should not be reused elsewhere. The censoring-corrected Case 1 exponent (3.09) is closer to values typical of obstructed links. The larger-sample models in the literature reach 4.97 to 7.5 dB [11], [15] because they describe the whole path. For practice, a planner without terrain data can expect most packets to arrive at SF11–12 and 14 dBm, but this safe setting wastes most of the energy at a typical stop. A short on-site sweep finds a much cheaper setting per node, path loss from distance alone should carry an uncertainty of about 11 dB, and the tiny tree could run on the node itself.

F. Ethics and reproducibility

No personal data were collected,. Site names and coordinates are withheld from the paper and the dashboard so that private properties cannot be located. Predictions and the dead-zone map are research outputs, not coverage guarantees for safety uses. The analysis notebook rebuilds every table and figure from the raw logs, and the dashboard reproduces the main classifier result.

VII. Limitations and Future Work

A. Limitations

This is a small study: 61 complete stops, only eight in Case 3, from two rooftop Stations of different estimated height, all in one season. Antenna gains were not measured, so path loss is apparent rather than absolute, and rooftop heights were estimated. GPS accuracy is about 5–10 m, SRTM resolution of 30 m limits the elevation features, and the Station-side log was unavailable. The dead-zone map applies a model trained on 61 stops to invented terrain and is an illustration only. Results describe this area and should not be generalised beyond the three cases.

B. Future work

More stops, cases and seasons would allow firmer cross-case validation. Station-side logging and measured antenna gains would give absolute path loss. Terrain-profile features computed along each path, such as obstruction depth [15] or knife-edge diffraction loss [3], should be tested with censoring-aware models and the same strict validation. Finally, the sweep could become an adaptive procedure in which a node tries settings from cheapest to most expensive and keeps the first that works.

VIII. Conclusion

Across 61 stops in three hill-terrain cases, delivery was governed by spreading factor and power, and a shallow tree on these two settings kept its accuracy on an unseen case while being among the cheapest models. More flexible models, including random forests and gradient boosting, did not transfer better. Path loss could not be predicted from distance or endpoint terrain better than by its mean (RMSE 11.1 dB), and a log-distance model held only in the short-range obstructed case. Per-stop settings could save a median 97% of transmit energy. Receiver censoring, unrecorded failed links and terrain mixing explain much of this. In hill terrain, what lies between the antennas matters more than the distance between them.

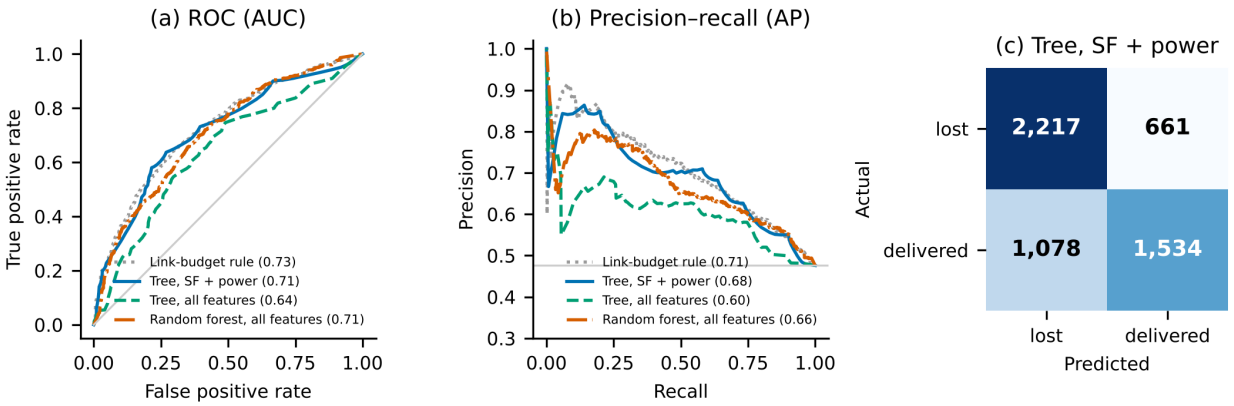


Fig. 21: (a) ROC and (b) precision–recall curves under grouped validation; (c) confusion matrix of the SF + power tree.

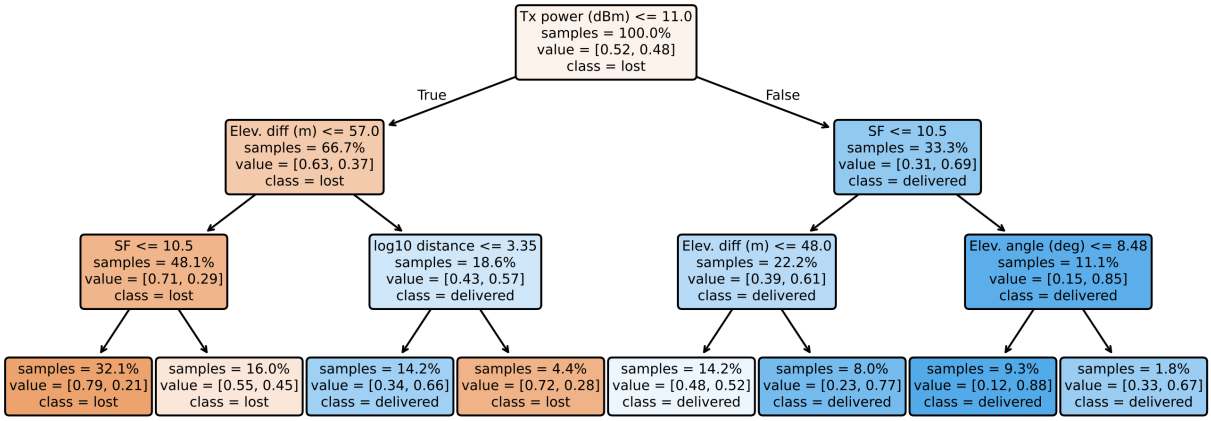


Fig. 22: Depth-3 delivery tree fitted to all data with all features (interpretation only).

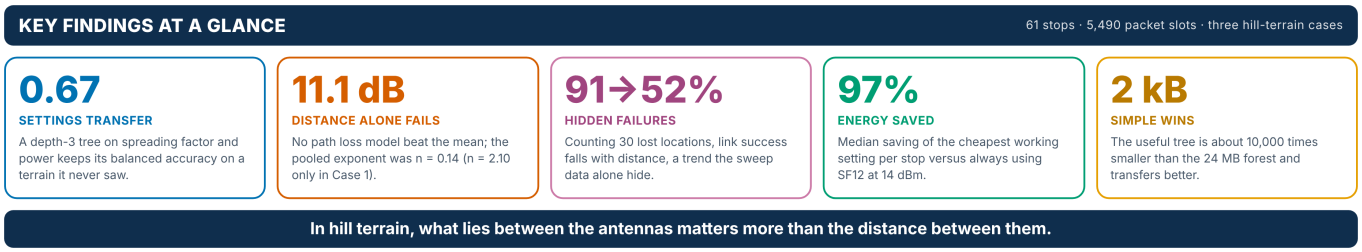


Fig. 23: Key findings at a glance.

Acknowledgment

I thank my module tutors at Softwarica College of IT and E-Commerce for their guidance and feedback. Generative AI tools assisted with code, figure layout and language editing; the experiment design, field measurements, analysis decisions and conclusions are my own, and every reported number was checked against the analysis notebook.

Data Availability

The raw Roamer logs, stop features, analysis notebook and dashboard code are available from the author on request. Site names and exact locations are withheld.

References

[1] T. S. Rappaport, *Wireless Communications: Principles and Practice*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2002.

[2] M. Hata, "Empirical formula for propagation loss in land mobile radio services," *IEEE Trans. Veh. Technol.*, vol. 29, no. 3, pp. 317–325, 1980.

[3] ITU-R, "Propagation by diffraction," Recommendation ITU-R P.526, International Telecommunication Union, Geneva, Switzerland.
[4] J. Petäjäjärvi, K. Mikhaylov, A. Roivainen, T. Hänninen, and M. Pettissalo, "On the coverage of LPWANs: Range evaluation and channel attenuation model for LoRa technology," in *Proc. 14th Int. Conf. ITS Telecommunications (ITST)*, 2015, doi:10.1109/ITST.2015.7377400.
[5] M. Rademacher, H. Linka, T. Horstmann, and M. Henze, "Path loss in urban LoRa networks: A large-scale measurement study," *arXiv preprint arXiv:2109.07768*, 2021.
[6] G. M. Bianco, A. Mejia-Aguilar, and G. Marrocco, "Radio wave propagation of LoRa systems in mountains for search and rescue operations," in *Proc. URSI General Assembly and Scientific Symposium (URSI GASS)*, 2020.
[7] A. Pokharel, P. Sapkota, D. Sapkota, S. Dahal, S. Karki, and R. K. Budhathoki, "Performance evaluation of LoRa technology for rural connectivity: An experimental analysis in Nepal," *arXiv preprint arXiv:2412.04563*, 2024.
[8] R. Bitterling, C. Nettersheim, J. Hees, and M. Rademacher, "A systematic sample size analysis of ML-based path loss prediction for LPWAN," *arXiv preprint arXiv:2608.11083*, 2026.
[9] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi:10.1023/A:1010933404324.
[10] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984.
[11] M. O. Ojo, I. Viola, S. Miretti, E. Martignani, S. Giordano, and M. Baratta, "A deep learning approach for accurate path loss prediction in LoRaWAN livestock monitoring," *Sensors*, vol. 24, no. 10, p. 2991, 2024, doi:10.3390/s24102991.
[12] J. Tobin, "Estimation of relationships for limited dependent variables," *Econometrica*, vol. 26, no. 1, pp. 24–36, 1958.
[13] C. Gustafson, T. Abbas, D. Bolin, and F. Tufvesson, "Statistical modeling and estimation of censored pathloss data," *arXiv preprint arXiv:1504.03977*, 2015.
[14] D. R. Roberts *et al.*, "Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure," *Ecography*, vol. 40, no. 8, pp. 913–929, 2017, doi:10.1111/ecog.02881.
[15] J. Ethier and M. Châteauvert, "Machine learning-based path loss modeling with simplified features," *arXiv preprint arXiv:2405.10006*, 2024.
[16] Semtech Corporation, "LoRa modulation basics," Application Note AN1200.22, 2015.
[17] A. Augustin, J. Yi, T. Clausen, and W. M. Townsley, "A study of LoRa: Long range & low power networks for the Internet of Things," *Sensors*, vol. 16, no. 9, p. 1466, 2016, doi:10.3390/s16091466.
[18] Nepal Telecommunications Authority, "Radio Frequency Policy," Kathmandu, Nepal, 2023.
[19] T. G. Farr *et al.*, "The Shuttle Radar Topography Mission," *Rev. Geophys.*, vol. 45, no. 2, RG2004, 2007, doi:10.1029/2005RG000183.
[20] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

APPENDIX A
FIELD USE AND ANALYSIS NOTEBOOK

Fig. 24 shows the Roamer carried during the field walk and its interface after a sweep, which displays delivery for each SF and power. Figs. 25–28 are excerpts of the executed analysis notebook. They cover data loading and checks, path loss regression, the delivery classifiers and the cost measurements. Cells printing GPS coordinates are omitted.

APPENDIX B
INTERACTIVE DASHBOARD

Fig. 29 shows the Gradio dashboard. It has seven tabs: overview, delivery explorer, path loss, model results, a model lab, the dead-zone map, and method and limits. It reads the raw logs with relative paths, removes coordinates, and runs locally or in Google Colab. The model lab reproduces the SF + power tree result live (balanced accuracy 0.68, AUC 0.71, LOCO 0.67). The dead-zone map is an illustration on synthetic terrain: where the tree's prediction and the line-of-sight layer disagree marks the information that endpoint features cannot capture.



(a) Roamer carried in a bag



(b) Roamer interface after a sweep

Fig. 24: Field use of the Roamer.

2. Load and combine the raw data

In [4]:
```
logs = []
for f in files:
    d = pd.read_csv(f)
    d['site'] = os.path.basename(f).split('_')[1]
    logs.append(d)
    print(f'{os.path.basename(f):26s} rows {len(d):5d}  ', d.status.value_counts().to_dict())
log = pd.concat(logs, ignore_index=True)
print('combined log:', log.shape)
log.groupby(['site', 'status']).size().unstack(fill_value=0)
```

```
site_A_roamer_log.csv      rows  1540  {'TIMEOUT': 967, 'OK': 563, 'NOACK': 10}
site_B_roamer_log.csv      rows  3262  {'OK': 1737, 'TIMEOUT': 1503, 'NOACK': 22}
site_C_roamer_log.csv      rows   376  {'OK': 197, 'TIMEOUT': 163, 'NOACK': 16}
site_D_roamer_log.csv      rows   368  {'TIMEOUT': 245, 'OK': 115, 'NOACK': 8}
combined log: (5546, 24)
```

Out[4]:

status	NOACK	OK	TIMEOUT
site			
A	10	563	967
B	22	1737	1503
C	16	197	163
D	8	115	245

In [5]:
```
sweeps = log[log.status.isin(['OK', 'TIMEOUT'])].copy()
sweeps['stop_key'] = sweeps.site + sweeps.stop_id.astype(int).astype(str)
slots = sweeps.groupby('stop_key').size()
complete = slots[slots == 90].index
raw = sweeps[sweeps.stop_key.isin(complete)].drop(columns='stop_key').reset_index(drop=True)
print('stops logged:', len(slots), '| complete stops:', len(complete), '| incomplete dropped:', int((slots != 90).sum()))

fails = (log[log.status == 'NOACK'][['site', 'stop_id', 'lat', 'lon']]
         .rename(columns={'stop_id': 'attempt_id'}).sort_values(['site', 'attempt_id']).reset_index(drop=True))
feat = pd.read_csv(os.path.join(DATA, 'stop_features.csv'))
stations = pd.read_csv(os.path.join(DATA, 'stations.csv'))
print('raw sweeps:', raw.shape, '| failed handshakes:', fails.shape, '| stop features:', feat.shape, '| stations:', stations.shape)
feat.head()
```

```
stops logged: 61 | complete stops: 61 | incomplete dropped: 0
raw sweeps: (5490, 24) | failed handshakes: (56, 4) | stop features: (61, 12) | stations: (4, 3)
```

Out[5]:

	stop_key	site	distance_to_station_m	station_ant_m	roamer_ant_m	stop_elev_m_srtm30	station_elev_m_srtm30	elev_diff_m	elev_angle_deg	terrain_category	land_cove
0	A13	A	236.0	2.0	1.3	1397	1372	25	5.88	mountain	fores
1	A14	A	202.0	2.0	1.3	1403	1372	31	8.53	mountain	mixe
2	A15	A	201.0	2.0	1.3	1403	1372	31	8.57	mountain	mixe
3	A16	A	182.0	2.0	1.3	1408	1372	36	10.98	mountain	fores
4	A18	A	229.0	2.0	1.3	1376	1372	4	0.83	mountain	mixe

3. Data checks

In [6]:
```
raw['stop_key'] = raw.site + raw.stop_id.astype(int).astype(str)
per_stop = raw.groupby('stop_key').size()
print('stops:', raw.stop_key.nunique(), '| every stop has 90 slots:', bool((per_stop == 90).all()))
print('slots:', len(raw), '| status counts:', raw.status.value_counts().to_dict())
print('received share: %.1f %%' % (100 * (raw.status == 'OK').mean()))
print('stops per site:', raw.groupby('site').stop_key.nunique().to_dict())
assert raw.stop_key.nunique() == 61 and (per_stop == 90).all() and len(raw) == 5490
assert set(raw.status.unique()) <= {'OK', 'TIMEOUT'}
assert raw.loc[raw.status == 'OK', 'rssi'].notna().all() and raw.loc[raw.status == 'TIMEOUT', 'rssi'].isna().all()
print('Cleaning checks passed: 61 complete stops, 5,490 slots, OK/TIMEOUT only, RSSI present exactly when OK.')
```

```
stops: 61 | every stop has 90 slots: True
slots: 5490 | status counts: {'TIMEOUT': 2878, 'OK': 2612}
received share: 47.6 %
stops per site: {'A': 17, 'B': 36, 'C': 4, 'D': 4}
Cleaning checks passed: 61 complete stops, 5,490 slots, OK/TIMEOUT only, RSSI present exactly when OK.
```

Fig. 25: Notebook: combining the site logs and data checks.

8. Path loss regression

In [15]:

```
from sklearn.linear_model import LinearRegression, RidgeCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import LeaveOneOut, LeaveOneGroupOut, cross_val_predict

y = d60.pl_median_14dbm.values; g = d60.case.values
MODELS = {
  'Mean only':                   (LinearRegression(), None),
  'Log-distance':                (LinearRegression(), ['logd']),
  '+ elev. diff. and angle':     (LinearRegression(), ['logd', 'elev_diff_m', 'elev_angle_deg']),
  '+ land cover (ridge)':        (make_pipeline(StandardScaler(), RidgeCV(alphas=np.logspace(-2, 3, 30))), ['logd', 'elev_diff_m', 'elev_angle_deg'] +
  'Regression tree, depth 3':    (DecisionTreeRegressor(max_depth=3, min_samples_leaf=5, random_state=0), ['logd', 'elev_diff_m', 'elev_angle_deg'] +
rows = []
for name, (m, cols) in MODELS.items():
    X = np.zeros((len(y), 1)) if cols is None else d60[cols].values
    loo = cross_val_predict(m, X, y, cv=LeaveOneOut()); lc = cross_val_predict(m, X, y, groups=g, cv=LeaveOneGroupOut())
    rows.append([name, rmse(y, loo), rmse(y, lc)] + [rmse(y[g == c], lc[g == c]) for c in (1, 2, 3)])
reg = pd.DataFrame(rows, columns=['model', 'LOO', 'LOCO', 'LOCO case1', 'LOCO case2', 'LOCO case3']).round(1)
print(reg.to_string(index=False)); RESULTS.update(rmse_mean=reg.LOO[0], rmse_logd_loco=reg.LOCO[1], rmse_terrain_loco=reg.LOCO[2])

print('\nWithin one case (LOO inside the case):')
for c in (1, 2):
    dd = d60[d60.case == c]; yy = dd.pl_median_14dbm.values
    r = [rmse(yy, cross_val_predict(LinearRegression(), X_, yy, cv=LeaveOneOut())) for X_ in [np.zeros((len(yy), 1)), dd[['logd']], dd[['logd', 'elev_
    print(f'  Case {c} ({len(dd)} stops): mean {r[0]:.1f} | log-distance {r[1]:.1f} | + elevation diff {r[2]:.1f}')
d51 = d60[d60.n_ok >= 10]; y51 = d51.pl_median_14dbm.values
print('Strict subset (>=10 received, %d stops): mean %.1f | log-distance %.1f' % (len(d51),
      rmse(y51, cross_val_predict(LinearRegression(), np.zeros((len(y51), 1)), y51, cv=LeaveOneOut())),
      rmse(y51, cross_val_predict(LinearRegression(), d51[['logd']], y51, cv=LeaveOneOut()))))
print('Manual line-of-sight check, delivery %:', (s.groupby('line_of_sight').delivery_rate.mean() * 100).round(0).to_dict())

fig, ax = plt.subplots(figsize=(W, 2.4)); h = .36; yy_ = np.arange(len(reg))
ax.barh(yy_ - h/2, reg.LOO, h, color='#0072B2', label='Leave-one-stop-out'); ax.barh(yy_ + h/2, reg.LOCO, h, color='#E69F00', label='Leave-one-case-ou
for i, r in reg.iterrows():
    ax.text(r.LOO + .3, i - h/2, f'{r.LOO:.1f}', va='center', fontsize=7); ax.text(r.LOCO + .3, i + h/2, f'{r.LOCO:.1f}', va='center', fontsize=7)
ax.set_yticks(yy_); ax.set_yticklabels(reg.model); ax.invert_yaxis(); ax.set_xlim(0, 21)
ax.set_xlabel('RMSE of median path loss (dB), lower is better'); ax.grid(axis='y', visible=False)
ax.legend(fontsize=7, frameon=False, loc='lower center', bbox_to_anchor=(.4, 1.0), ncol=2)
fig.savefig(f'{FIG}/fig4_regression_rmse.png'); plt.show()
```

```
                 model  LOO  LOCO  LOCO case1  LOCO case2  LOCO case3
             Mean only 11.1  11.4         8.4        12.9         9.1
          Log-distance 11.3  13.0         9.8        14.8         9.5
+ elev. diff. and angle 11.4  15.8        11.2        16.3        20.9
   + land cover (ridge) 11.1  15.7         8.4        17.1        19.8
Regression tree, depth 3 11.2  15.3         9.5        16.9        16.8

Within one case (LOO inside the case):
  Case 1 (16 stops): mean 8.7 | log-distance 8.0 | + elevation diff 7.2
  Case 2 (36 stops): mean 12.6 | log-distance 12.9 | + elevation diff 11.9
Strict subset (>=10 received, 51 stops): mean 10.6 | log-distance 10.9
Manual line-of-sight check, delivery %: {'blocked': 27.0, 'clear': 76.0, 'partly blocked': 53.0}
```

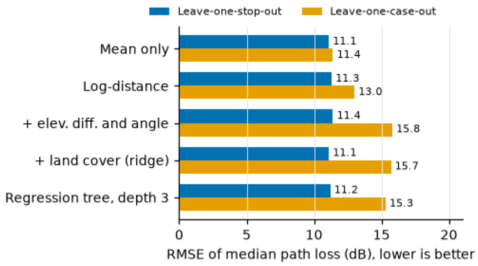


Fig. 26: Notebook: path loss regression.

Other models, metrics, ROC and confusion matrix

In [19]:

```
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (precision_score, recall_score, f1_score, confusion_matrix, roc_curve,
                             precision_recall_curve, average_precision_score)
SFP, ALLF = SETS['SF + power'], SETS['+ land cover']
mk_tree = lambda: DecisionTreeClassifier(max_depth=3, min_samples_leaf=90, random_state=0)
MODELS = {'Tree, SF + power': (mk_tree(), SFP), 'Tree, all features': (mk_tree(), ALLF),
          'Random forest, all features': (RandomForestClassifier(300, random_state=0, n_jobs=-1), ALLF),
          'Logistic regression, SF + power': (make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)), SFP),
          'Logistic regression, all features': (make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)), ALLF),
          'Gradient boosting, all features': (HistGradientBoostingClassifier(random_state=0), ALLF)}
PROBS = {'Link-budget rule': prob}; LOCO = {'Link-budget rule': np.nan}
for name, (mdl, cols) in MODELS.items():
    PROBS[name] = cross_val_predict(mdl, p[cols], yp, groups=GS, cv=gkf, method='predict_proba')[:, 1]
    LOCO[name] = balanced_accuracy_score(yp, cross_val_predict(mdl, p[cols], yp, groups=GC, cv=LeaveOneGroupOut(), method='predict_proba')[:, 1] > .5)
rows = []
for name, pr in PROBS.items():
    yhat = (pred > 0) if name == 'Link-budget rule' else (pr > .5)
    tn, fp, fn, tp = confusion_matrix(yp, yhat).ravel()
    rows.append([name, balanced_accuracy_score(yp, yhat), precision_score(yp, yhat), recall_score(yp, yhat), f1_score(yp, yhat),
                 roc_auc_score(yp, pr), average_precision_score(yp, pr), LOCO[name], tn, fp, fn, tp])
full = pd.DataFrame(rows, columns=['model', 'bal_acc', 'precision', 'recall', 'F1', 'AUC', 'AP', 'LOCO_bal_acc', 'TN', 'FP', 'FN', 'TP'])
print(full.round(3).to_string(index=False))
RESULTS.update(tree_precision=full.precision[1], tree_recall=full.recall[1], tree_f1=full.F1[1], gb_auc=full.AUC[6], gb_loco=full.LOCO_bal_acc[6],
               lr_sfp_loco=full.LOCO_bal_acc[4], linkbudget_recall=full.recall[0])

fig, ax = plt.subplots(1, 3, figsize=(W * 2.05, 2.45), gridspec_kw=dict(width_ratios=[1, 1, .82]))
for n, c, ls in [('Link-budget rule', '#999999', ':'), ('Tree, SF + power', '#0072B2', '-'),
                 ('Tree, all features', '#009E73', '--'), ('Random forest, all features', '#D55E00', '-.')]:
    fpr, tpr, _ = roc_curve(yp, PROBS[n]); ax[0].plot(fpr, tpr, c=c, ls=ls, lw=1.3, label=f'{n} ({roc_auc_score(yp, PROBS[n]):.2f})')
    pr_, rc_, _ = precision_recall_curve(yp, PROBS[n]); ax[1].plot(rc_, pr_, c=c, ls=ls, lw=1.3, label=f'{n} ({average_precision_score(yp, PROBS[n]):.
ax[0].plot([0, 1], [0, 1], c='#ccc', lw=.8); ax[0].set(xlabel='False positive rate', ylabel='True positive rate', title='(a) ROC (AUC)')
ax[1].axhline(yp.mean(), c='#ccc', lw=.8); ax[1].set(xlabel='Recall', ylabel='Precision', title='(b) Precision-recall (AP)', ylim=(.3, 1.02))
for a in ax[:2]: a.legend(fontsize=5.2, frameon=False, loc='lower right')
cm = confusion_matrix(yp, PROBS['Tree, SF + power'] > .5); ax[2].imshow(cm, cmap='Blues'); ax[2].grid(False)
for i in range(2):
    for j in range(2): ax[2].text(j, i, f'{cm[i, j]:,}', ha='center', va='center', fontweight='bold', color='white' if cm[i, j] > 1500 else 'black')
ax[2].set_xticks([0, 1], ['lost', 'delivered']); ax[2].set_yticks([0, 1], ['lost', 'delivered']); ax[2].set(xlabel='Predicted', ylabel='Actual', title
fig.tight_layout(); fig.savefig(f'{FIG}/fig9_roc_pr_cm.png'); plt.show()
```

model	bal_acc	precision	recall	F1	AUC	AP	LOCO_bal_acc	TN	FP	FN	TP
Link-budget rule	0.538	0.496	0.970	0.656	0.727	0.706	NaN	302	2576	78	2534
Tree, SF + power	0.679	0.699	0.587	0.638	0.713	0.684	0.671	2217	661	1078	1534
Tree, all features	0.620	0.625	0.528	0.573	0.641	0.600	0.501	2052	826	1233	1379
Random forest, all features	0.646	0.633	0.618	0.625	0.707	0.664	0.553	1942	936	998	1614
Logistic regression, SF + power	0.677	0.672	0.637	0.654	0.734	0.706	0.654	2066	812	949	1663
Logistic regression, all features	0.674	0.664	0.644	0.654	0.726	0.697	0.556	2026	852	929	1683
Gradient boosting, all features	0.680	0.661	0.674	0.667	0.745	0.723	0.576	1974	904	851	1761

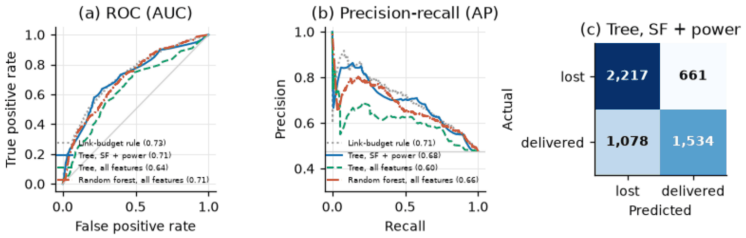


Fig. 27: Notebook: delivery classifiers, metrics, ROC and confusion matrix.

10. Cheapest setting per stop

In [20]:
```
mw = {2: 10 ** .2, 8: 10 ** .8, 14: 10 ** 1.4}
gs = p.groupby(['stop_key', 'case', 'sf', 'pwr']).ok.sum().reset_index()
gs['cost'] = 2.0 ** (gs.sf - 7) * gs.pwr.map(mw)
best = gs[gs.ok >= 4].sort_values('cost').groupby('stop_key').first()
ms = p.groupby('stop_key').case.first().to_frame().join(best[['sf', 'pwr', 'cost']])
full_cost = 32 * mw[14]
rows = []
for c, dd in list(ms.groupby('case')) + [('All', ms)]:
    sv = dd[dd.sf.notna()]
    rows.append([c, f'{len(sv)}/{len(dd)}', int(((sv.sf <= 9) & (sv.pwr <= 8)).sum()), int(((sv.sf >= 11) & (sv.pwr == 14)).sum()),
                 round(100 * (1 - sv.cost / full_cost).median())])
mt = pd.DataFrame(rows, columns=['case', 'stops with a working setting', 'low cost (SF<=9, <=8 dBm)', 'needed SF11-12 at 14 dBm', 'median energy savir
print(mt.to_string(index=False))
RESULTS.update(min_served=int(ms.sf.notna().sum()), min_saving=float(mt.iloc[-1, -1]))
```

```
case stops with a working setting  low cost (SF<=9, <=8 dBm)  needed SF11-12 at 14 dBm  median energy saving %
   1                        13/17                          4                         2                      97
   2                        34/36                         13                         3                      97
   3                          8/8                          2                         2                      92
 All                        55/61                         19                         7                      97
```

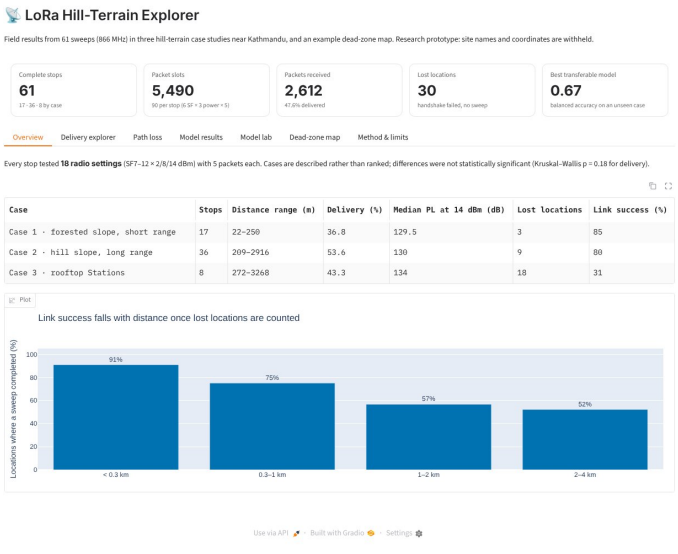
11. Computational cost

In [21]:
```
import time, pickle

cols_r = ['logd', 'elev_diff_m', 'elev_angle_deg'] + LC
def cost(name, mdl, X, y_):
    ft = []; pt = []
    for _ in range(5):
        t0 = time.perf_counter(); mdl.fit(X, y_); ft.append(time.perf_counter() - t0)
        t0 = time.perf_counter(); mdl.predict(X); pt.append(time.perf_counter() - t0)
    return dict(model=name, fit_ms=1e3 * np.median(ft), predict_us_per_sample=1e6 * np.median(pt) / len(X), size_kB=len(pickle.dumps(mdl)) / 1024)
cc = pd.DataFrame([
    cost('Log-distance (OLS)', LinearRegression(), d60[['logd']], d60.pl_median_14dbm),
    cost('Ridge, all features', make_pipeline(StandardScaler(), RidgeCV(alphas=np.logspace(-2, 3, 30))), d60[cols_r], d60.pl_median_14dbm),
    cost('Regression tree, depth 3', DecisionTreeRegressor(max_depth=3, min_samples_leaf=5, random_state=0), d60[cols_r], d60.pl_median_14dbm),
    cost('RF regressor, 300 trees', RandomForestRegressor(300, random_state=0, n_jobs=1), d60[cols_r], d60.pl_median_14dbm),
    cost('Tree, SF + power', mk_tree(), p[SFP], yp), cost('Tree, all features', mk_tree(), p[ALLF], yp),
    cost('RF classifier, 300 trees', RandomForestClassifier(300, random_state=0, n_jobs=1), p[ALLF], yp)]).round(2)
print(cc.to_string(index=False))
print('RF classifier is %.0fx larger than the SF + power tree' % (cc.size_kB.iloc[-1] / cc.size_kB.iloc[4]))
```

```
                model  fit_ms  predict_us_per_sample  size_kB
     Log-distance (OLS)    1.32                   9.68     0.59
     Ridge, all features    4.16                  13.92     1.42
Regression tree, depth 3    1.24                   9.72     1.83
 RF regressor, 300 trees  282.25                 431.54  1574.60
       Tree, SF + power    1.84                   0.13     2.40
     Tree, all features    2.97                   0.14     2.47
RF classifier, 300 trees  885.72                  14.62 24397.74
RF classifier is 10166x larger than the SF + power tree
```

Fig. 28: Notebook: cheapest setting and computational cost.



(a) Overview (b) Dead-zone map, 3-D view

Fig. 29: Gradio dashboard (no site names or coordinates).